

01/10/2025

Chapitre 3 : Les Tests

Programmation

UE-3 : Informatique

Licence 2^{ème} Année

Parcours Sciences pour l'ingénieur

CHAPUIS Yves André
UNIVERSITE DE STRASBOURG

Cours librement inspiré du « Cours de Python » de Patrick FUCHS
et Pierre POULAIN, Université de Paris, 2021

1. Introduction

C'est quoi un test en programmation ?

Les tests sont un élément essentiel à tout langage informatique si on veut lui donner un peu de complexité car ils permettent à l'ordinateur de prendre des décisions. Pour cela, Python utilise l'instruction `if` ainsi qu'une comparaison que nous avons abordée au chapitre précédent.

Voici un premier exemple :

```
1 x = 2
2 if x == 2:
3     print("Le test est vrai ! ")
```

Le test est vrai !

Et un second :

```
1 x = "souris"
2 if x == "tigre":
3     print("Le test est vrai !")
```

Il y a plusieurs remarques à faire concernant ces deux exemples :

- Dans le premier exemple, le test étant **VRAI**, l'instruction `print("Le test est vrai !")` est exécutée.
- Dans le second exemple, le test est **FAUX** et rien n'est affiché.

A retenir

L'instruction conditionnelle **if** permet de ne faire exécuter un bloc d'instructions lorsqu'une certaine condition est remplie (VRAI → On exécute ; FAUX → Pas d'exécution)

En langage Python, l'instruction **if** se termine toujours par le caractère **deux-points** « : ».

Ne pas oublier que les blocs d'instructions dans les tests doivent forcément être **indentés** comme dans le cas des *fonctions*. **L'indentation** indique la portée des instructions à exécuter si le test est VRAI.

2. Comparaisons

Python est capable d'effectuer toute une série de comparaisons entre le contenu de deux variables. Ces comparaisons se basent sur des opérateurs simples et couramment utilisés en mathématique, et dont la syntaxe Python est donnée dans le [TABLEAU 1](#) ci-dessous.

TABLEAU 1. Opérateurs de comparaison en syntaxe Python

Syntaxe Python	Signification
<code>==</code>	égal à
<code>!=</code>	différent de
<code>></code>	supérieur à
<code>>=</code>	supérieur ou égal à
<code><</code>	inférieur à
<code><=</code>	inférieur ou égal à

Observez les exemples suivants avec des nombres entiers.

1	<code>x = 5</code>
2	<code>x == 5</code>
True	
1	<code>x = 8</code>
2	<code>x > 10</code>
False	
1	<code>x = 8</code>
2	<code>x < 10</code>
True	

Python renvoie la valeur **True** si la comparaison est vraie et **False** si elle est fausse. True et False sont des *booléens* (un nouveau type de variable).

Faites bien attention à ne pas confondre l'opérateur d'affectation « = » qui affecte une valeur à une variable et l'opérateur de comparaison « == » qui compare les valeurs de deux variables.

Vous pouvez également effectuer des comparaisons sur des chaînes de caractères.

1	<code>animal = "tigre"</code>
2	<code>animal == "tig"</code>
False	

```
1 animal = "tigre"
2 animal != "tig"
True
```

```
1 animal = "tigre"
2 animal == "tigre"
True
```

Dans le cas des chaînes de caractères, a priori seuls les tests « == » et « != » ont un sens. En fait, on peut aussi utiliser les opérateurs « < », « > », « <= » et « >= ». Dans ce cas, l'ordre alphabétique est pris en compte, par exemple :

```
1 "a" < "b"
True
```

"a" est inférieur à "b" car le caractère a est situé avant le caractère b dans l'ordre alphabétique. En fait, c'est l'ordre ASCII des caractères qui est pris en compte (à chaque caractère correspond un code numérique), on peut donc aussi comparer des caractères spéciaux (comme « # » ou « ~ ») entre eux.

Enfin, on peut comparer des chaînes de caractères de plusieurs caractères :

```
1 "ali" < "alo"
True
```

```
1 "abb" < "ada"
True
```

Dans ce cas, Python compare les deux chaînes de caractères, caractère par caractère, de la gauche vers la droite (le premier caractère avec le premier, le deuxième avec le deuxième, etc). Dès qu'un caractère est différent entre l'une et l'autre des deux chaînes, il considère que la chaîne la plus petite est celle qui présente le caractère ayant le plus petit code ASCII (les caractères suivants de la chaîne de caractères sont ignorés dans la comparaison), comme dans l'exemple "abb" < "ada" ci-dessus.

3. Tests à plusieurs cas

Parfois, il est pratique de tester si la condition est Vraie (True) ou si elle est Fausse (False) dans une même instruction **if**. Plutôt que d'utiliser deux instructions **if**, on peut se servir des instructions **if** et **else** :

```
1 x = 2
2 if x == 2:
3     print("Le test est vrai !")
4 else:
5     print("Le test est faux !")
```

Le test est vrai !

```
1 x = 3
2 if x == 2:
3     print("Le test est vrai !")
4 else:
5     print("Le test est faux !")
```

Le test est faux !

On peut utiliser une série de tests dans la même instruction **if**, notamment pour tester plusieurs valeurs d'une même variable.

Par exemple, on se propose de tirer au sort une base d'ADN puis d'afficher le nom de cette dernière. Dans le code suivant, nous utilisons l'instruction **random.choice(liste)** qui renvoie un élément choisi au hasard dans une liste. L'instruction **import random** sera vue plus tard, admettez pour le moment qu'elle est nécessaire.

```
1 import random
2
3 base = random.choice(["a", "t", "c", "g"])
4
5 print(f"La variable base affectée au hasard est : {base}")
6
7 if base == "a":
8     print("Choix d'une adénine")
9 elif base == "t":
10    print("Choix d'une thymine")
11 elif base == "c":
12    print("Choix d'une cytosine")
13 elif base == "g":
14    print("Choix d'une guanine")
```

La variable base affectée au hasard est : t
Choix d'une thymine

Dans cet exemple, Python teste la première condition, puis, si et seulement si elle est fausse, teste la deuxième et ainsi de suite ...

Le code correspondant à la première condition vérifiée est exécuté puis Python sort du **bloc d'instructions de l'if** et l'on utilise **elif** pour tester les autres cas.

4. Tests multiples

4.1. Rappel sur les opérateurs booléens

Les tests multiples permettent de tester plusieurs conditions en même temps en utilisant des opérateurs booléens. Les deux opérateurs les plus couramment utilisés sont le OU (OR *in English*) et le ET (AND *in English*).

Voici un petit rappel sur le fonctionnement de l'opérateur OU sur le [TABLEAU 2](#).

TABLEAU 2. Fonctionnement de l'opérateur boolean OU (OR)

Condition 1	Opérateur	Condition 2	Résultat
Vrai	OU	Vrai	Vrai
Vrai	OU	Faux	Vrai
Faux	OU	Vrai	Vrai
Faux	OU	Faux	Faux

Et le fonctionnement de l'opérateur AND sur le [TABLEAU 3](#).

TABLEAU 3. Fonctionnement de l'opérateur boolean ET (AND)

Condition 1	Opérateur	Condition 2	Résultat
Vrai	ET	Vrai	Vrai
Vrai	ET	Faux	Faux
Faux	ET	Vrai	Faux
Faux	ET	Faux	Faux

4.2. Utilisation des opérateurs booléens pour réaliser des tests multiples

En Python, on utilise le mot réservé **and** pour l'opérateur ET et le mot réservé **or** pour l'opérateur OU. Respectez bien la casse des opérateurs and et or qui, en Python, s'écrivent en minuscule. En voici un exemple d'utilisation :

```

1 x = 2
2 y = 2
3
4 if x == 2 and y == 2:
5     print("Le test est vrai !")

```

Le test est vrai !

Notez que le même résultat serait obtenu en utilisant deux instructions **if imbriquées** :

```

1 x = 2
2 y = 2
3
4 if x == 2:
5     if y == 2:
6         print("Le test est vrai !")

```

Le test est vrai !

Vous pouvez aussi tester directement l'effet de ces opérateurs à l'aide de **True** et **False** (attention à respecter la casse).

```
1 True or False
```

True

Enfin, on peut utiliser l'opérateur logique de négation **not** qui inverse le résultat d'une condition :

```
1 not True
```

False

```
1 not False
```

True

```
1 not (True and True)
```

False

5. Tests de valeur sur des *float*

Lorsque l'on souhaite tester la valeur d'une variable de type float, le premier réflexe serait d'utiliser l'opérateur d'égalité comme :

```
1 1/10 == 0.1
True
```

Toutefois, nous vous le déconseillons formellement. Pourquoi ? Python stocke les valeurs numériques des floats sous forme de nombres flottants (d'où leur nom!), et cela mène à certaines limitations¹. Observez l'exemple suivant :

```
1 (3 - 2.7) == 0.3
False
```

```
1 3 - 2.7
0.2999999999999998
```

Nous voyons que le résultat de l'opération $3 - 2.7$ n'est pas exactement 0.3 d'où le False.

En fait, ce problème ne vient pas de Python, mais plutôt de la manière dont un ordinateur traite les nombres flottants (comme un rapport de nombres binaires). Ainsi certaines valeurs de float ne peuvent être qu'approchées. Une manière de s'en rendre compte est d'utiliser l'écriture formatée en demandant l'affichage d'un grand nombre de décimales :

```
1 0.3
0.3

1 f"{0.3:.5f}"
'0.30000'

1 f"{0.3:.60f}"
'0.29999999999999988897769753748434595763683319091796875000000'

1 f"{3 - 2.7:.60f}"
'0.299999999999999822364316059974953532218933105468750000000000'
```

On observe que lorsqu'on tape 0.3, Python affiche une valeur arrondie. En réalité, le nombre réel 0.3 ne peut être qu'approché lorsqu'on le code en nombre flottant. Il est donc essentiel d'avoir cela en tête lorsque l'on effectue un test.

1. <https://docs.python.org/fr/3/tutorial/floatingpoint.html>

Conseil

Pour les raisons évoquées ci-dessus, il ne faut surtout pas tester si un float est égal à une certaine valeur. La bonne pratique est de vérifier si un float est compris dans un intervalle avec une certaine précision.

Si on appelle cette précision delta, on peut procéder ainsi :

Méthode 1

```
1 delta = 0.0001
2 var = 3.0 - 2.7
3 0.3 - delta < var < 0.3 + delta
```

True

Méthode 2

```
1 abs(var - 0.3) < delta
```

True

Ici on teste si **var** est compris dans l'intervalle $[0.3 - \text{delta}, 0.3 + \text{delta}]$.

Les deux méthodes mènent à un résultat strictement équivalent :

- Dans la première méthode, la *ligne 3* est intuitive car elle ressemble à un encadrement mathématique.
 - Dans la deuxième méthode, la *ligne 1* utilise la fonction « valeur absolue » *abs()* et est plus compacte.
-

6. Structure algorithmique

6.1. Logigramme (ou organigramme) mode d'emploi

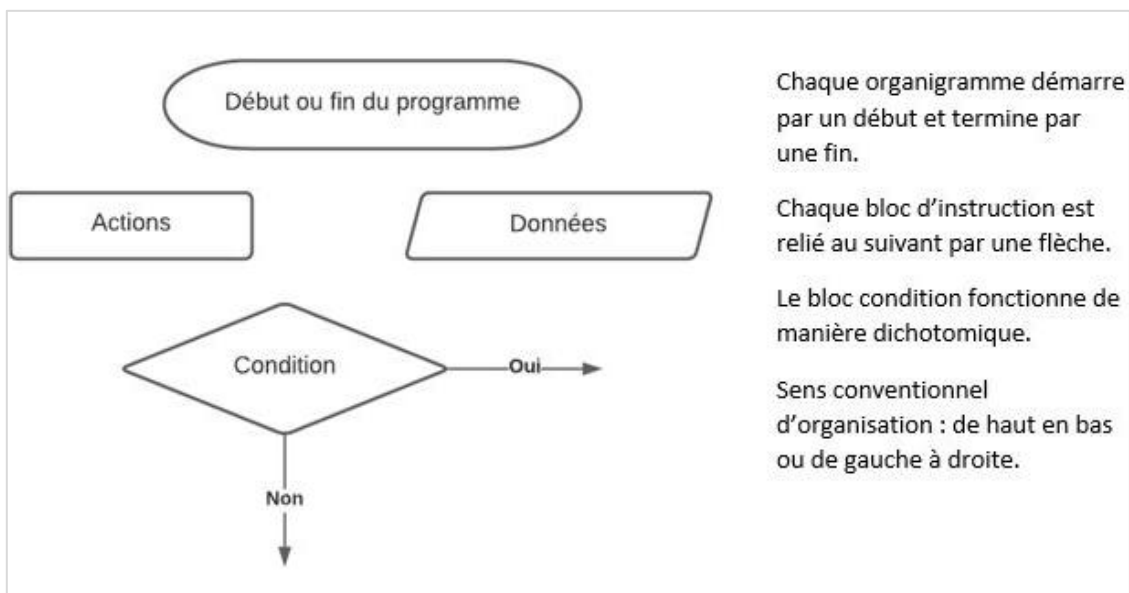
Avant d'écrire un programme, il est essentiel de **décrire la logique de traitement** de manière claire et structurée.

Le **logigramme** (ou organigramme) est un outil graphique qui permet de **représenter visuellement** la séquence des opérations et les **tests logiques** d'un algorithme.

6.2. Éléments de base du logigramme

Explique les éléments de base :

Élément	Symbole	Signification
Début / Fin	○ ou ovale	Indique le point d'entrée ou de sortie de l'algorithme
Action / Instruction	□ (rectangle)	Représente une opération (calcul, affectation, affichage...)
Test / Condition	◇ (losange)	Sert à poser une question, dont la réponse oriente le flux ("Oui / Non")
Flèches	→	Indiquent le sens de l'exécution des étapes



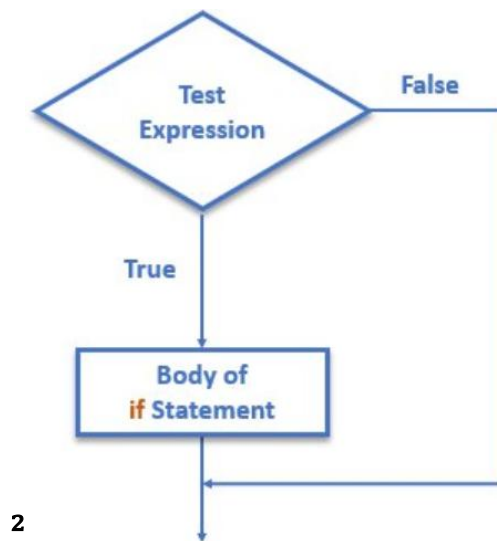
6.3. Condition de test **if ...**

La syntaxe en Python d'une instruction conditionnelle est, assez communément,

```
if test:
    # instructions
```

Si le test est vrai les instructions situées après **if** et indentées (*c'est-à-dire espacées vers la droite*) sont exécutées, pas dans le cas contraire.

Le traitement d'une condition de test *if ...* peut se représenter par le logigramme :



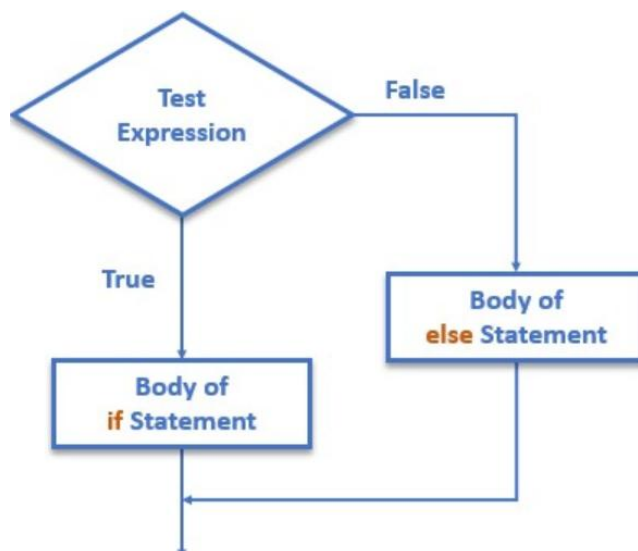
Essayer par exemple:

```
n=float(input("Entrer un nombre: "));
if n<100:
    print("Ce n'est pas bien grand...")
```

6.4. Condition de test *if ... else*

Plus généralement, on peut compléter la structure conditionnelle précédente en indiquant aussi quoi faire si la condition n'est pas vraie *else* en anglais et en python.

Le traitement d'une condition de test *if ... else* peut se représenter par le logigramme :



La syntaxe en Python devient :

```
if test:
    # instructions
else:
    # AUTres instructions
```

Essayer par exemple:

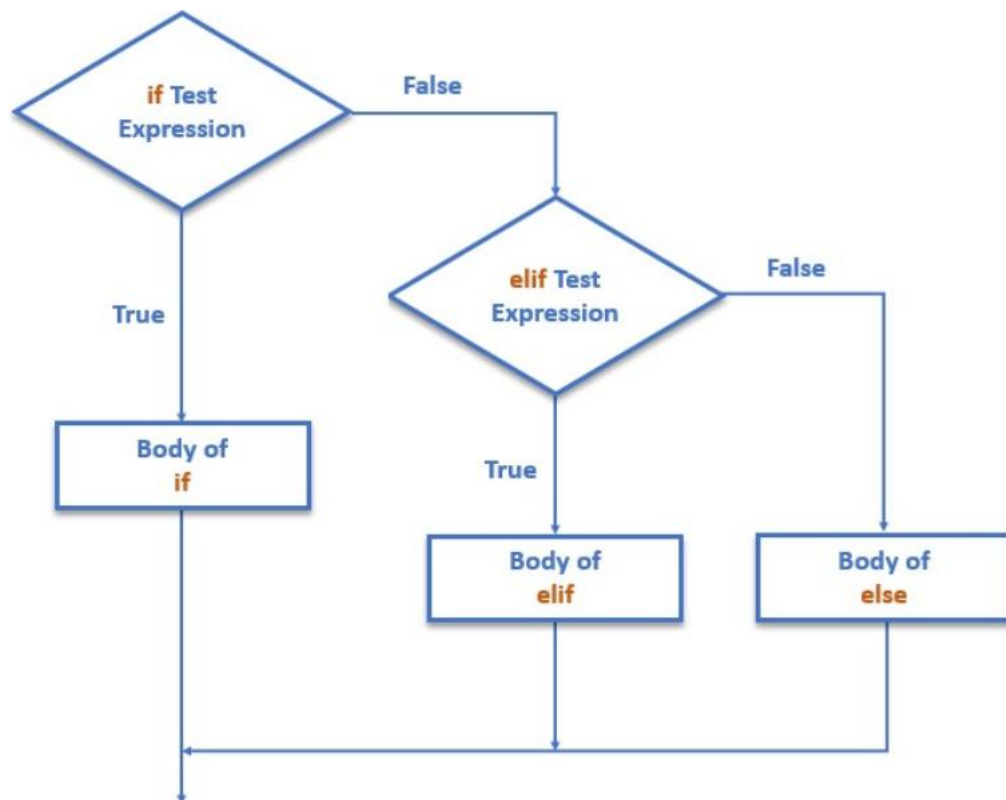
```
n=float(input("Entrer un nombre: "));
if n<100:
    print("Ce n'est pas bien grand...")
else:
    print("C'est assez grand...")
```

6.5. Traitement d'une structure de test *if ... elif*

En Python, la structure de test *if ... else* est utilisée quand tu veux tester **une seule condition de test**, avec une alternative si elle est fausse.

Mais si l'on veut tester **plusieurs conditions successives de test**, il faut utiliser *elif* (abréviation de *else if*), soit une structure de test : *if ... elif*

Le traitement d'une condition de test *if ... elif* peut se représenter par le logigramme :



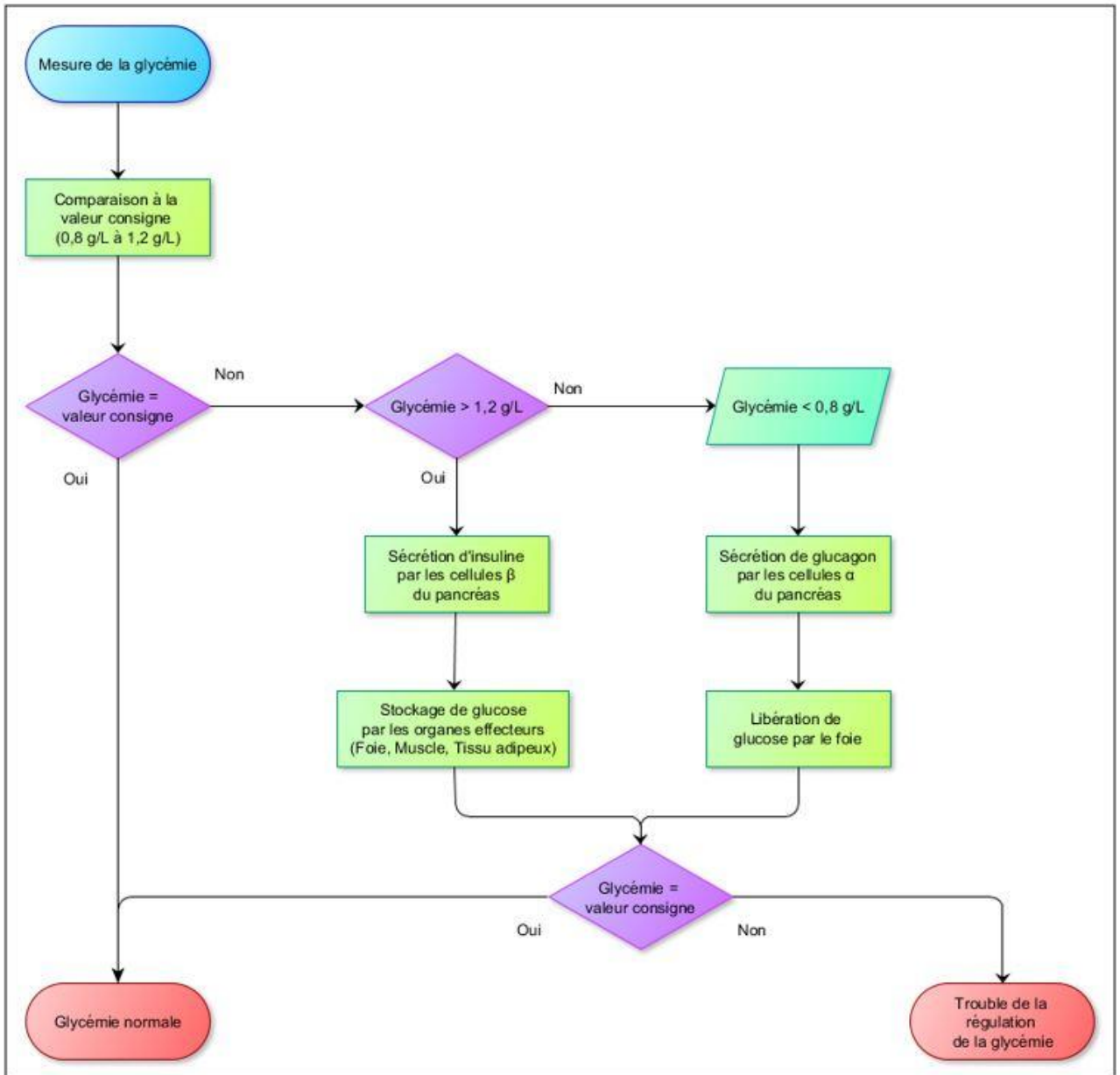
La syntaxe en Python devient dans l'exemple :

```
age = 12

if age >= 13 and age <= 19:
    print("Person is a teenager")
elif age >= 0 and age < 13:
    print("Person is a prepubescent")
else:
    print("Person is an adult")
```

6.6. Exemple de logigramme d'une structure de test

Ci-dessous, on donne un exemple de logigramme de programmation appliqué à une notion travaillée en analyse biologique dans le cas d'un test de glycémie.



7. Exercices

7.1. Parité d'un nombre entier

Écrire un programme qui détermine si un nombre entier n est **pair** ou **impair**.

7.2. Signe d'un nombre

Déterminer si un nombre x est **positif**, **négatif** ou **nul**.

7.3. Affichage conditionnel d'une appréciation

Écrire un programme qui affiche une appréciation selon la note d'un élève (sur 20) :

- $[16-20] \rightarrow$ "Très bien"
- $[12-15] \rightarrow$ "Bien"
- $[8-11] \rightarrow$ "Passable"
- $[0-7] \rightarrow$ "Insuffisant"

7.4. Accès à un tarif préférentiel

Une personne bénéficie d'un tarif réduit si :

- elle a moins de 18 ans,
- ou si elle a plus de 60 ans,
- ou si elle possède une carte étudiante.

7.5. Calcul du salaire avec prime

Un salarié reçoit :

- un **salaire de base** (variable `salaire_base`),
- une **prime** de 10% si son **ancienneté** est ≥ 5 ans,
- et une **prime supplémentaire** de 5% si sa **note d'évaluation** ≥ 15 .

Données :

```
salaire_base = 2500
anciennete = 6
note = 16
```

Calculer le **salaire total** à verser.

7.6. Classification d'un triangle

Étant donnés trois côtés a , b , c , déterminer :

- Si le triangle est **équilatéral** (3 côtés égaux),
- **isocèle** (2 côtés égaux),
- **scalène** (tous différents),
- ou **non valide** (la somme de deux côtés $\leq$ au troisième).

Données :

```
a, b, c = 5, 5, 8
```

Calculer le **salaire total** à verser.

Correction

7.1. Parité d'un nombre entier

```
n = 7

if n % 2 == 0:
    print("Le nombre est pair.")
else:
    print("Le nombre est impair.")
```

Le nombre est impair

7.2. Signe d'un nombre

```
x = -5

if x > 0:
    print("Le nombre est positif.")
elif x < 0:
    print("Le nombre est négatif.")
else:
    print("Le nombre est nul.")
```

Le nombre est négatif.

7.3. Affichage conditionnel d'une appréciation

```
note = 14

if note >= 16:
    print("Très bien")
elif note >= 12:
    print("Bien")
elif note >= 8:
    print("Passable")
else:
    print("Insuffisant")
```

Bien

7.4. Accès à un tarif préférentiel

```
age = 25
carte_etudiante = True

if age < 18 or age > 60 or carte_etudiante:
    print("Tarif réduit accordé")
else:
    print("Tarif normal")
```

Tarif réduit accordé

7.5. Calcul du salaire avec prime

```
salaire_base = 2500
anciennete = 6
note = 16

prime = 0

if anciennete >= 5:
    prime += 0.10
    if note >= 15: # condition imbriquée
        prime += 0.05

salaire_total = salaire_base * (1 + prime)
print("Salaire total :", salaire_total, "€")
```

Salaire total : 2875.0 €

7.6. Classification d'un triangle

```
a, b, c = 5, 5, 8

if a + b > c and a + c > b and b + c > a:
    if a == b == c:
        print("Triangle équilatéral")
    elif a == b or a == c or b == c:
        print("Triangle isocèle")
    else:
        print("Triangle scalène")
else:
    print("Triangle non valide")
```

Triangle isocèle